Claude Multi-Account Fine Tuning

Claude Toolkit

Bash toolkit for running **multiple Claude Code accounts on one host** while keeping conversation history, memory, todos, plans, plugins, and settings **unified** across them. Each account keeps only what must stay private (credentials and per-account state); everything else lives in a single shared store and is reachable from every account via symlinks.

It can also **share the whole Claude Code ecosystem with OpenCode** — every plugin's Skills, MCP servers, and the user `CLAUDE.md` — via one command (`claude-opencode-sync`). See **OpenCode_Integration.md**.

The companion deep-dive is `Claude_Multi_Account_Fine_Tuning.md` (also rendered as `.html` and `.pdf` siblings).

What you get

- One alias per account: `claude1`, `claude2`, ... (or custom names like `claudework`) — each launches Claude Code with its own `CLAUDE_CONFIG_DIR`.
- A single shared store at `~/.claude-shared/` holding `projects/`, `todos/`, `tasks/`, `plans/`, `history.jsonl`, `settings.json`, `plugins/`, `CLAUDE.md`,

and friends. Switching accounts is just a shell alias change; the conversation history, project memory, and installed plugins follow you.
- **Cross-account session resume**: the `projects` / session index inside `.claude.json` is deep-merged across every account on each launch. Sessions created under `claude1` show up in `claude2`'s `--resume` list the next time you launch it — and vice versa. Auth keys (`userID`, `oauthAccount`, `firstStartTime`, `claudeCodeFirstTokenDate`) stay per-account.
- Per-account directories are mostly symlinks pointing into the shared store. Only `.credentials.json` and `mcp-needs-auth-cache.json` stay strictly private; `.claude.json` is per-account but its non-auth contents (project index, MCP server status, UX state, caches) sync across accounts.

Requirements

`bash`, `rsync`, `jq`, `awk`. Optional: `pandoc` (+ `weasyprint` or `wkhtmltopdf` or headless `chromium`) for regenerating the PDF/HTML docs.

Verified on Linux and macOS. On macOS, install via Homebrew:

```
brew install jq rsync gawk pandoc weasyprint
```

Install

Pick the path that matches your host:

A. Existing accounts (host already has ≥1 `~/.claude-*` dir)

```
git clone <this repo> claude-toolkit
cd claude-toolkit
bash scripts/install.sh         # symlinks scripts to ~/.local/
        bin, writes

        # managed alias file, sources from rc files,

        # runs unify on existing accounts, regenerates
                                # docs if pandoc is present.
exec $SHELL -l                  # reload shell so aliases load
claude-list-accounts            # confirm setup
```

B. Clean-slate host (Claude Code installed but no accounts yet)

```
git clone <this repo> claude-toolkit
cd claude-toolkit
bash scripts/claude-bootstrap.sh --count 2 --yes     # provision
        claude1, claude2
exec $SHELL -l
claude1 /login                  # authenticate each account once
claude2 /login
claude-list-accounts
```

`--aliases personal,work` instead of `--count` for custom names; `--dir-of NAME=PATH` to override an alias's config dir.

Both scripts are **idempotent** — safe to re-run after pulling updates.

macOS notes

- macOS ships bash 3.2; the scripts auto-re-exec under Homebrew bash (`brew install bash`) when needed.
- All rc-file edits target `~/.zshrc` only on Darwin (zsh is the default interactive shell).

Daily commands

Command	Purpose
`claude-list-accounts`	Tabular status: alias, config dir, creds present, link health.
`claude-add-account`	Add a new account. Interactive, or `--alias NAME --dir PATH --yes`.
`claude-remove-account --alias NAME`	Drop an alias; archive (default) or `--delete` its dir.
`claude-unify`	Re-merge state into the shared store. No args → auto-detects all `~/.claude-*` dirs.
`claude-sync-state`	Fast JSON-level sync of `.claude.json` project/session index across accounts (no rsync). Run automatically by the `cma_run` alias wrapper before/after every claude launch.
`claude-bootstrap`	Clean-slate provisioning on a fresh host with no accounts logged in yet.
`claude-rollback`	Restore the `.preunify.*` backups and move the shared store aside.
`claude-export-docs`	Regenerate `Claude_Multi_Account_Fine_Tuning.{html,pdf}` from the markdown.
`claude-opencode-sync`	Expose all Claude plugin Skills + MCP servers + `CLAUDE.md` to a host-installed OpenCode. See OpenCode_Integration.md.
`claude-providers`	Create/refresh Claude Code aliases for **other LLM providers** (DeepSeek, Groq, Mistral, GLM, …) from your keys file, pointed at each provider's strongest model. `sync`/`list`/`show`/`remove`/`add`. See Provider Aliases User Guide.

After adding an account, log in once:

```
claude3 /login
```

Provider aliases (other LLMs)

`claude-providers` turns every LLM API key in your keys file (`~/api_keys.sh`) into its own Claude Code alias, pointed at that provider's strongest model — **fully dynamically**, with no hardcoded provider list, base URLs, or model IDs (everything is derived from your keys + the models.dev catalog, validated by the bundled `LLMsVerifier` submodule).

```
claude-providers sync
        # discover keys -> create one alias per provider
source ~/.local/share/claude-multi-account/aliases.sh
claude-providers list          # alias, provider, transport,
        strong/fast model
deepseek                       # launch Claude Code on DeepSeek's
        best model
```

- **Native** providers (Anthropic-compatible) run `claude` directly; **router** providers (OpenAI-compatible / Gemini) go through claude-code-router.
- Provider config dirs (`~/.claude-prov-<id>`) reuse all your plugins + history and are **excluded from account detection**, so existing `claudeN` accounts and `claude-add-account` are completely unaffected.
- Secrets never leave the keys file — the launch wrapper injects them per session; nothing is written to the repo or the alias file.

See the Provider Aliases User Guide for overrides, verification, and the documented `/color` limitation.

Layout after unification

```
~/.claude-shared/                    # single source of truth
  projects/  todos/  tasks/  plans/  file-history/  paste-cache/
  plugins/  shell-snapshots/  session-env/  telemetry/  sessions/
  backups/  cache/  CLAUDE.md  history.jsonl  settings.json
  stats-cache.json

~/.claude-<account>/                 # per-account dir, mostly
symlinks
  .credentials.json                  # PRIVATE — account-locked
  .claude.json                       # PRIVATE — per-account state
  mcp-needs-auth-cache.json          # PRIVATE — auth state
  projects -> ~/.claude-shared/projects
  ...                                # every shared item is a
symlink

~/.local/share/claude-multi-account/
  aliases.sh                         # managed alias file (sourced
from rc files)
```

How cross-account session sync works

Claude Code stores a `projects.<path>` map inside `~/.claude-<account>/.claude.json` that holds `lastSessionId`, MCP server status, project-level memory pointers, and other per-project state. Without intervention, account A's projects are invisible to account B even though the underlying JSONL session transcripts live in the shared `projects/` directory.

The toolkit fixes this in two complementary layers:

1. **At unify time** — `claude-unify` deep-merges every account's `.claude.json`, biasing rightmost-wins on scalar conflicts and recursively unioning the `projects` subtree. Each account's auth-private keys (`userID`, `oauthAccount`, `firstStartTime`, `claudeCodeFirstTokenDate`) are written back untouched.
2. **At runtime** — the `cma_run` shell function (installed automatically by `install.sh` into the alias file) calls `claude-sync-state pull` before every claude launch and `claude-sync-state push` after exit. This is a cheap `jq` deep-merge — typically tens of milliseconds — so sessions created in one account are visible to others on the very next launch.

If you ever need to refresh manually:

```
claude-sync-state all       # merge across every detected account
claude-sync-state pull ~/.claude-<account>   # before launching
        account
claude-sync-state push ~/.claude-<account>   # after exiting
        account
```

Auth keys never cross. Verified via the `test_sessions.sh` test suite (30 assertions covering byte-stable idempotency, identity preservation, cross-account visibility, and corrupt-file resilience).

OpenCode integration

`claude-opencode-sync` translates the portable contents of every installed Claude plugin into a host-installed OpenCode's config:

- **Skills** → `skills.paths` (every plugin `SKILL.md` folder)
- **MCP servers** → `mcp{}` (local + remote, deduped, `${CLAUDE_PLUGIN_ROOT}` expanded; a safe no-auth subset is enabled, the rest configured-but-disabled)
- `CLAUDE.md` → `instructions[]`

```
claude-opencode-sync --dry-run --stats   # preview
claude-opencode-sync
        # apply (additive, backs up prior config)
```

On the reference host this wires **1,000+ skills** and **110+ MCP servers** into OpenCode while preserving its existing providers and servers. Full guide, diagrams, enable policy, and auth steps: **OpenCode_Integration.md**.

Testing

```
bash scripts/tests/run-all.sh                  # all sandboxed
        suites
bash scripts/tests/run-all.sh lib unify sessions opencode   #
        subset by suffix

bash scripts/tests/verify_opencode_live.sh     # live proof vs
        real OpenCode
bash scripts/tests/run-proof.sh                # both + dated
        evidence bundle
```

Tests use a sandboxed `$HOME` via `mktemp` — your real `~/.claude*` state is never touched. `test_sessions.sh` verifies cross-account session/memory visibility with physical proofs (sha256 byte-hashes, project key intersection, auth-key preservation); `test_opencode.sh` covers the OpenCode sync (MCP translation, dedup, enable gating, idempotency, config preservation). `run-proof.sh` writes inspectable evidence to `scripts/tests/proof/`.

Rollback

```
claude-rollback
# or equivalently:
claude-unify --rollback
```

Restores every `.preunify.<timestamp>` backup created during unification and moves the shared store aside to `~/.claude-shared.removed.<timestamp>`.

License

See repository.